

Operating Systems Lab

Week 7: Contiguous Memory Allocation

1. First Fit, Best Fit, and Worst Fit

Code:

```
#include <stdio.h>

void firstFit(int blockSize[], int m, int processSize[], int n)
{
    int allocation[n];    // stores process allocation
    int blockAllocated[m]; // stores whether allocated or not

    // init allocation = -1
    for (int i = 0; i < n; i++)
        allocation[i] = -1;
    // init allocated = 0
    for (int j = 0; j < m; j++)
        blockAllocated[j] = 0;

    // choosing suitable block
    for (int i = 0; i < n; i++)
    {
        for (int j = 0; j < m; j++)
        {
            if (blockSize[j] >= processSize[i] && blockAllocated[j] ==
0)
            {
                allocation[i] = j;
                blockAllocated[j] = 1;
                break;
            }
        }
    }

    printf("\nFirst Fit\n");
    printf("Process No.\tProcess Size\tBlock No.\n");
    for (int i = 0; i < n; i++)
    {
        printf("%d\t%d\t", i + 1, processSize[i]);
        if (allocation[i] != -1)
```

```

        printf("%d\n", allocation[i] + 1);
    else
        printf("Not Allocated\n");
    }
}

void bestFit(int blockSize[], int m, int processSize[], int n)
{
    int allocation[n];
    int blockAllocated[m];

    for (int i = 0; i < n; i++)
        allocation[i] = -1;
    for (int j = 0; j < m; j++)
        blockAllocated[j] = 0;

    for (int i = 0; i < n; i++)
    {
        int bestIdx = -1; // index of smallest fitting block
        for (int j = 0; j < m; j++)
        {
            if (blockSize[j] >= processSize[i] && blockAllocated[j] ==
0)
            {
                if (bestIdx == -1 || blockSize[j] < blockSize[bestIdx])
                    bestIdx = j;
            }
        }

        if (bestIdx != -1)
        {
            allocation[i] = bestIdx;
            blockAllocated[bestIdx] = 1;
        }
    }

    printf("\nBest Fit\n");
    printf("Process No.\tProcess Size\tBlock No.\n");
    for (int i = 0; i < n; i++)
    {
        printf("%d\t\t%d\t\t", i + 1, processSize[i]);
        if (allocation[i] != -1)
            printf("%d\n", allocation[i] + 1);
        else
            printf("Not Allocated\n");
    }
}

```

```

}

void worstFit(int blockSize[], int m, int processSize[], int n)
{
    int allocation[n];
    int blockAllocated[m];

    for (int i = 0; i < n; i++)
        allocation[i] = -1;
    for (int j = 0; j < m; j++)
        blockAllocated[j] = 0;

    for (int i = 0; i < n; i++)
    {
        int worstIdx = -1; // index of largest found block
        for (int j = 0; j < m; j++)
        {
            if (blockSize[j] >= processSize[i] && blockAllocated[j] ==
0)
            {
                if (worstIdx == -1 || blockSize[j] >
blockSize[worstIdx])
                    worstIdx = j;
            }
        }

        if (worstIdx != -1)
        {
            allocation[i] = worstIdx;
            blockAllocated[worstIdx] = 1;
        }
    }

    printf("\nWorst Fit\n");
    printf("Process No.\tProcess Size\tBlock No.\n");
    for (int i = 0; i < n; i++)
    {
        printf("%d\t\t%d\t\t", i + 1, processSize[i]);
        if (allocation[i] != -1)
            printf("%d\n", allocation[i] + 1);
        else
            printf("Not Allocated\n");
    }
}

int main()

```

```
{
    printf("Contiguous Memory Allocation\n");
    printf("Royden Miranda 1WA24CS240\n\n");

    int m, n;

    printf("Enter number of memory blocks: ");
    scanf("%d", &m);

    int blockSize[m], blockCopy[m];
    printf("Enter sizes of %d memory blocks:\n", m);
    for (int i = 0; i < m; i++)
        scanf("%d", &blockSize[i]);

    printf("Enter number of processes: ");
    scanf("%d", &n);

    int processSize[n];
    printf("Enter sizes of %d processes:\n", n);
    for (int i = 0; i < n; i++)
        scanf("%d", &processSize[i]);

    firstFit(blockSize, m, processSize, n);
    bestFit(blockSize, m, processSize, n);
    worstFit(blockSize, m, processSize, n);

    return 0;
}
```

Output:

Contiguous Memory Allocation
Royden Miranda 1WA24CS240

Enter number of memory blocks: 5
Enter sizes of 5 memory blocks:
100 500 200 300 600
Enter number of processes: 4
Enter sizes of 4 processes:
212 417 112 426

First Fit

Process No.	Process Size	Block No.
1	212	2
2	417	5
3	112	3
4	426	Not Allocated

Best Fit

Process No.	Process Size	Block No.
1	212	4
2	417	2
3	112	3
4	426	5

Worst Fit

Process No.	Process Size	Block No.
1	212	5
2	417	2
3	112	4
4	426	Not Allocated